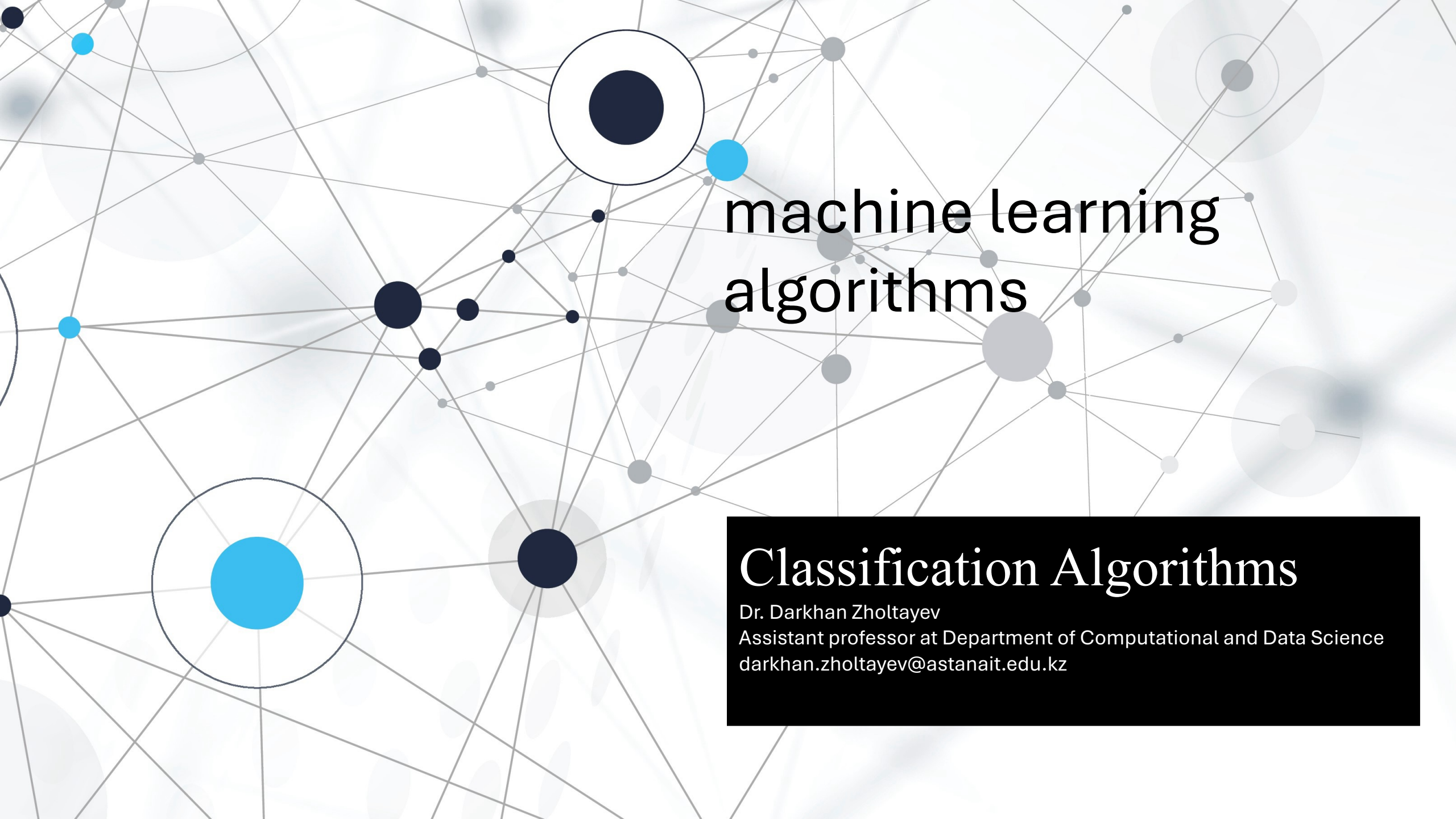


The background of the slide features a complex network diagram. It consists of numerous nodes of varying sizes and colors (dark blue, light blue, and grey) connected by thin, light grey lines. Some nodes are highlighted with larger, concentric circles. The overall aesthetic is modern and technical, suggesting a focus on data science or machine learning.

machine learning algorithms

Classification Algorithms

Dr. Darkhan Zholtayev

Assistant professor at Department of Computational and Data Science

darkhan.zholtayev@astanait.edu.kz

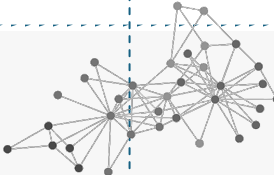
Outline

Decision trees,

K-Nearest Neighbors (KNN),

Support Vector Machines (SVM)

Their concepts, strengths & weaknesses

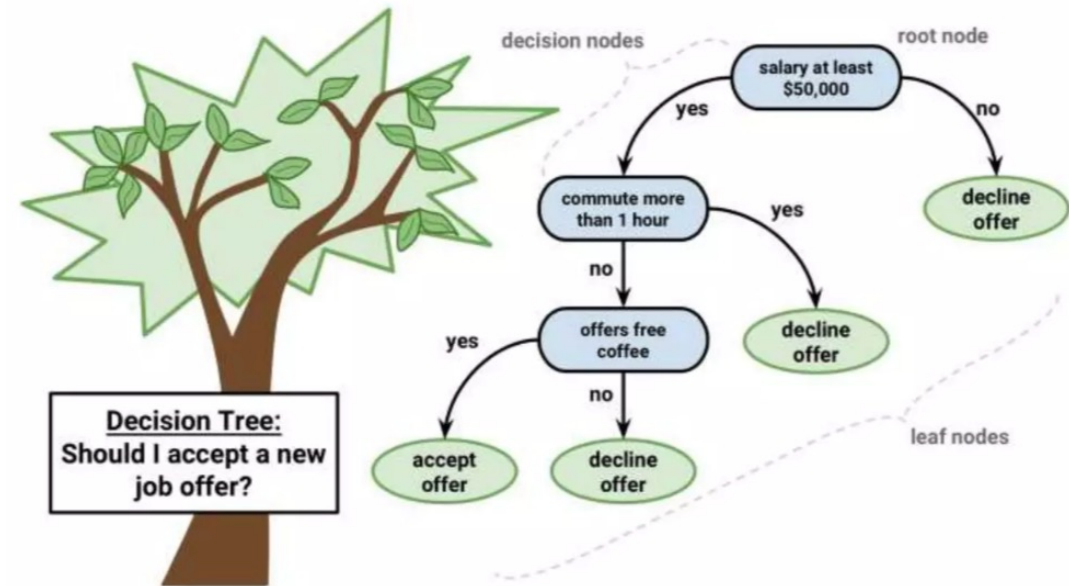


Section one



Decision trees

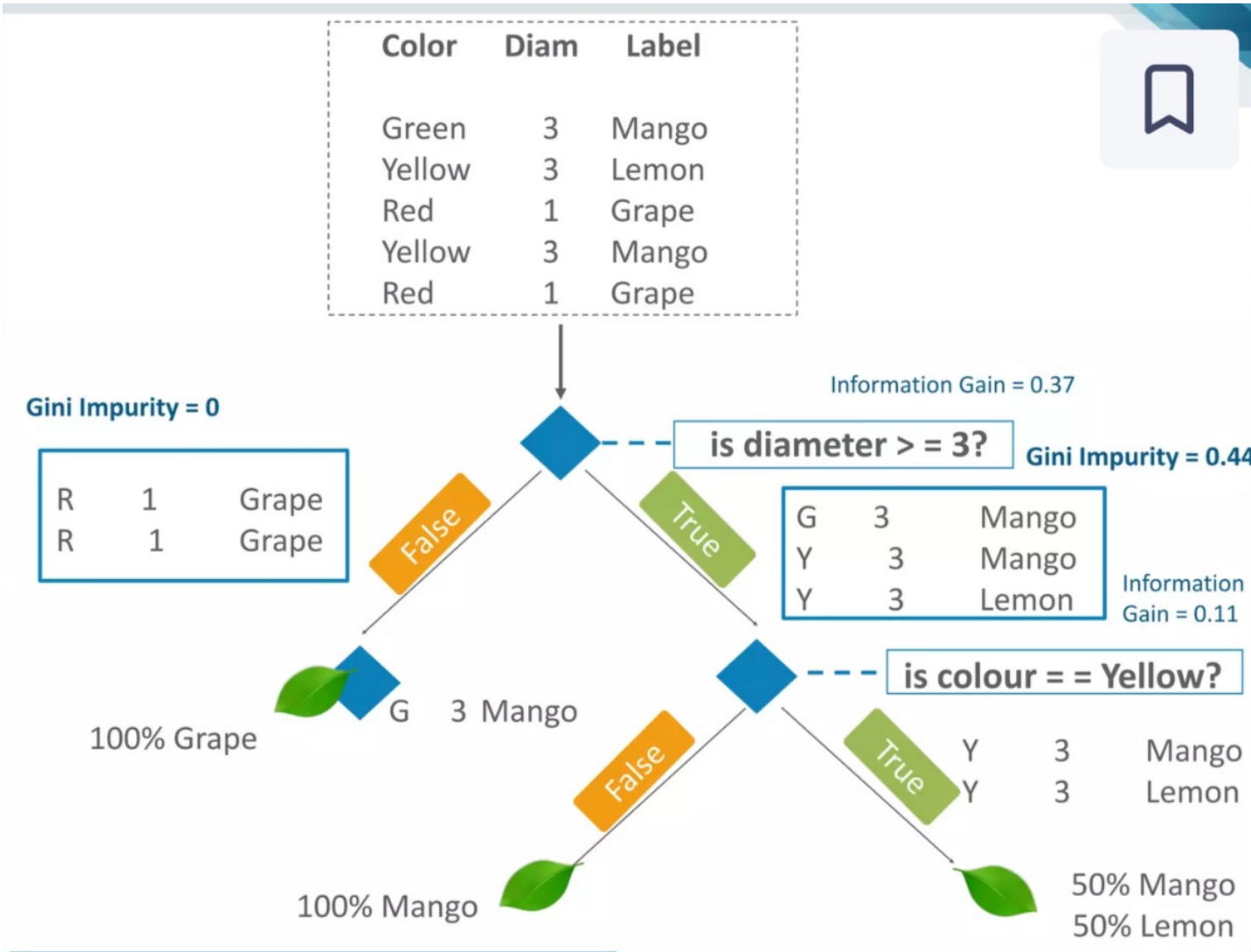
"A decision tree is a graphical representation of all the possible solutions to a decision based on certain conditions"



Dataset

Colour	Diameter	Label
Green	3	Mango
Yellow	3	Mango
Red	1	Grape
Red	1	Grape
Yellow	3	Lemon





Decision Tree



Decision tree



How Does A Tree Decide Where To Split?

Gini Index

The measure of impurity (or purity) used in building decision tree in CART is Gini Index

Chi Square

It is an algorithm to find out the statistical significance between the differences between sub-nodes and parent node



Information Gain

The information gain is the decrease in entropy after a dataset is split on the basis of an attribute. Constructing a decision tree is all about finding attribute that returns the highest information gain

Reduction in Variance

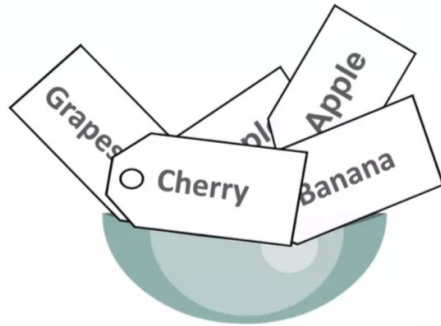
Reduction in variance is an algorithm used for continuous target variables (regression problems). The split with lower variance is selected as the criteria to split the population



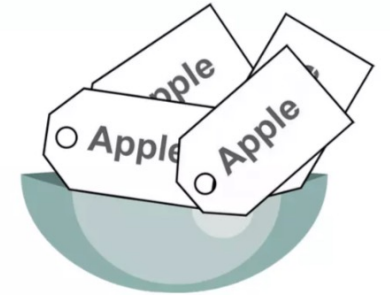
Purity and impurity



Impurity $\neq 0$

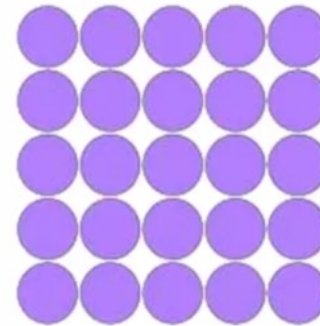


Impurity = 0

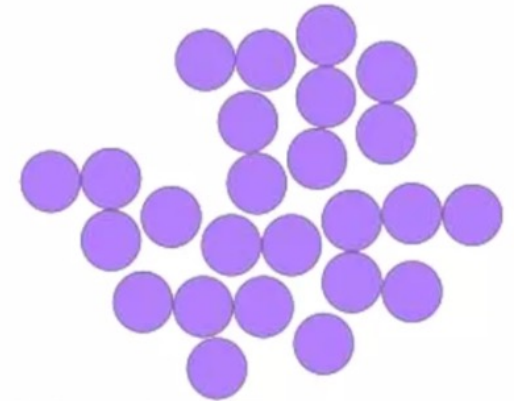


Entropy

- Defines randomness in the data
- **Entropy** is just a metric which measures the impurity or
- The first step to solve the problem of a decision tree



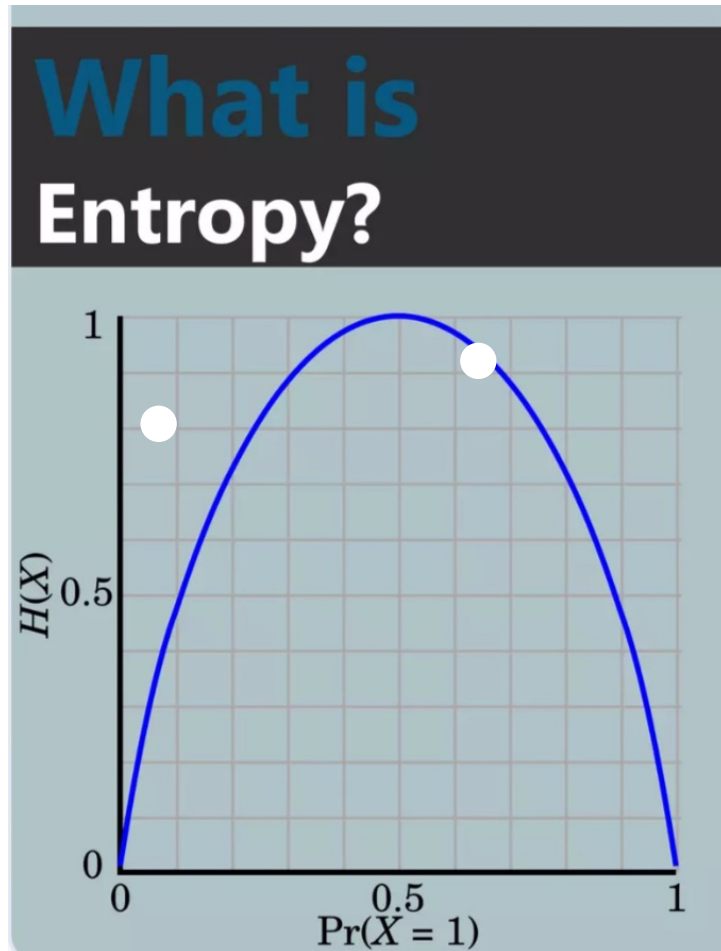
Low Entropy



High Entropy



Entropy 2



$$E(S) = -P(\text{Yes}) \log_2 P(\text{Yes})$$

When $P(\text{Yes}) = P(\text{No}) = 0.5$ ie YES + NO = Total Sample(S)

$$E(S) = 0.5 \log_2 0.5 - 0.5 \log_2 0.5$$

$$E(S) = 0.5(\log_2 0.5 - \log_2 0.5)$$

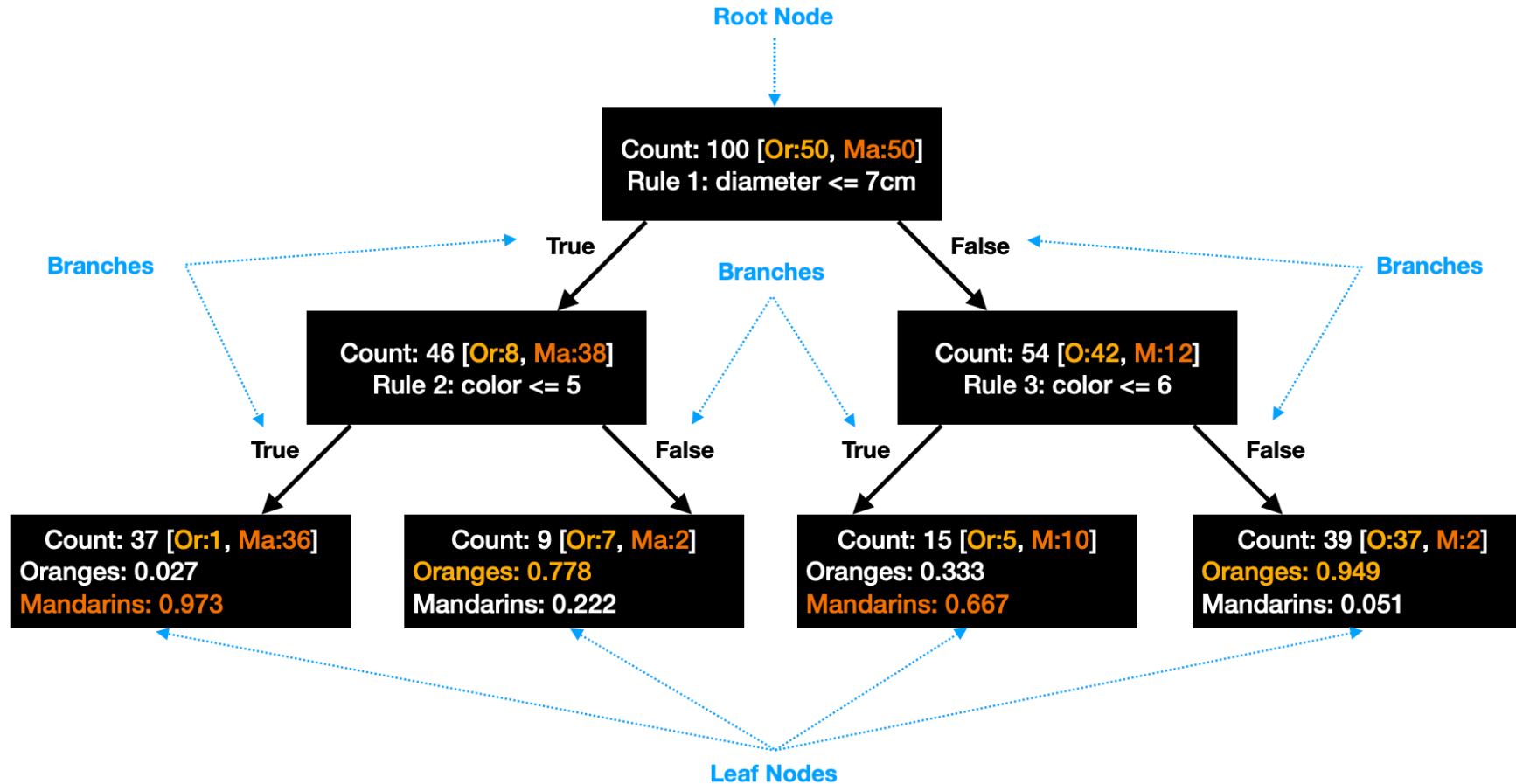
$$E(S) = 1$$

If all the data points in S belong to the same class (pure node), the entropy is 0 because there is no disorder.

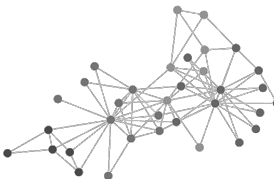
If the data points are evenly distributed across the classes (maximum disorder), the entropy is high.



An example



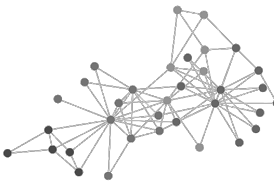
Jackson, L. (2018, November 29). *CART classification and regression trees for clean but powerful models*. Towards Data Science. Retrieved from <https://medium.com/towards-data-science/cart-classification-and-regression-trees-for-clean-but-powerful-models-cc89e60b7a85>



Formula

$$\textit{Gini Impurity} = 1 - \textit{Gini} = 1 - \sum_{i=1}^n p_i^2$$

where p_i is the fraction of items in the class i .



Splitting using Gini

- Let's assume a node is split into two child nodes. The Gini impurity for the split is calculated as:

$$Gini_{\text{split}} = \frac{n_{\text{left}}}{n_{\text{total}}} \times Gini_{\text{left}} + \frac{n_{\text{right}}}{n_{\text{total}}} \times Gini_{\text{right}}$$

Where:

n_{left} and n_{right} are the number of samples in the left and right child nodes.

n_{total} is the total number of samples in the parent node.

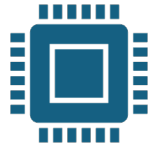
$Gini_{\text{left}}$ and $Gini_{\text{right}}$ are the Gini impurities of the left and right child nodes, respectively.

Key steps in training



1. Start with the Root Node:

- The root node contains the entire training dataset.
- The algorithm checks all possible features and all possible split points within those features to determine the best split.
- The best split is the one that reduces the impurity (e.g., Gini impurity for classification) the most.



2. Evaluate Splits:

- For each feature, the algorithm considers different possible values (thresholds) to split the data into two child nodes.
- For example, if a feature is Age, the split might be $\text{Age} \leq 30$.
- For classification, the algorithm calculates the Gini impurity (or entropy) for the two resulting nodes and chooses the split that minimises the weighted average of the child node impurities.



3. Recursive Splitting:

- After the first split, the algorithm repeats the process for each child node, treating each as a new dataset.
- The recursion continues until one of the following stopping criteria is met:
 - **Maximum Depth:** The tree reaches a predefined maximum depth (e.g., 5 levels).
 - **Minimum Samples per Node:** A node has fewer than a specified number of samples (e.g., fewer than 5 samples).
 - **Pure Node:** A node is “pure,” meaning all the samples in the node belong to the same class.



4. End Condition:

- Once the stopping criterion is met, the recursion ends. Each leaf node of the tree represents a final decision, containing either the majority class (in classification) or the average value (in regression).

Example of calculation

Suppose you have a node with the following class distribution:

Class 0: 80 samples

Class 1: 20 samples

First, calculate the proportions for each class:

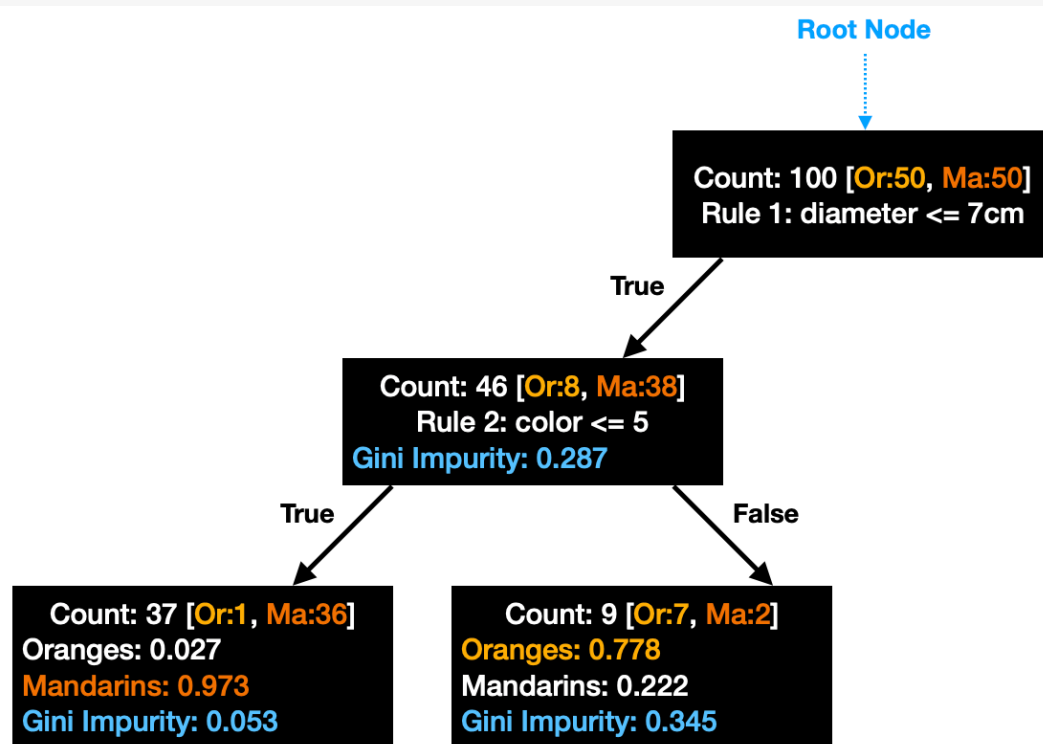
$$p_0 = \frac{80}{100} = 0.8, \quad p_1 = \frac{20}{100} = 0.2$$

Then, compute the Gini impurity:

$$Gini = 1 - (p_0^2 + p_1^2) = 1 - (0.8^2 + 0.2^2) = 1 - (0.64 + 0.04) = 1 - 0.68 = 0.32$$

Thus, the Gini impurity for this node is **0.32**.

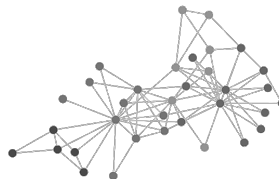
Gini impurity calculation



The weighted sum of the Gini Impurity for the two child nodes is:

$$\text{Gini Impurity} = \frac{37}{46} * 0.053 + \frac{9}{46} * 0.345 = 0.110$$

Important note: If the best weighted Gini Impurity for the two child nodes is **not lower** than Gini Impurity for the parent node, you should not split the parent node any further.



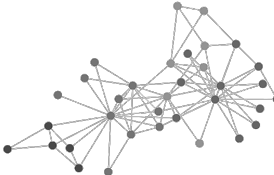
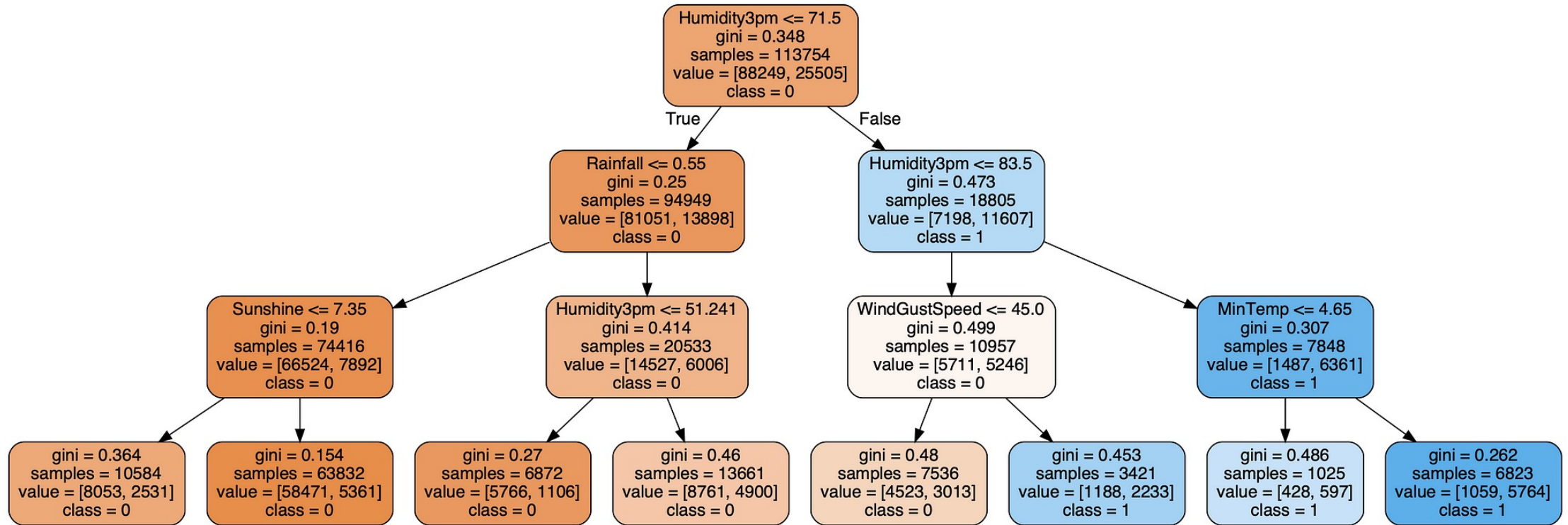
$$\textit{Entropy} = - \sum_{i=1}^n p_i \log_2(p_i)$$

Entropy is the
another
approach

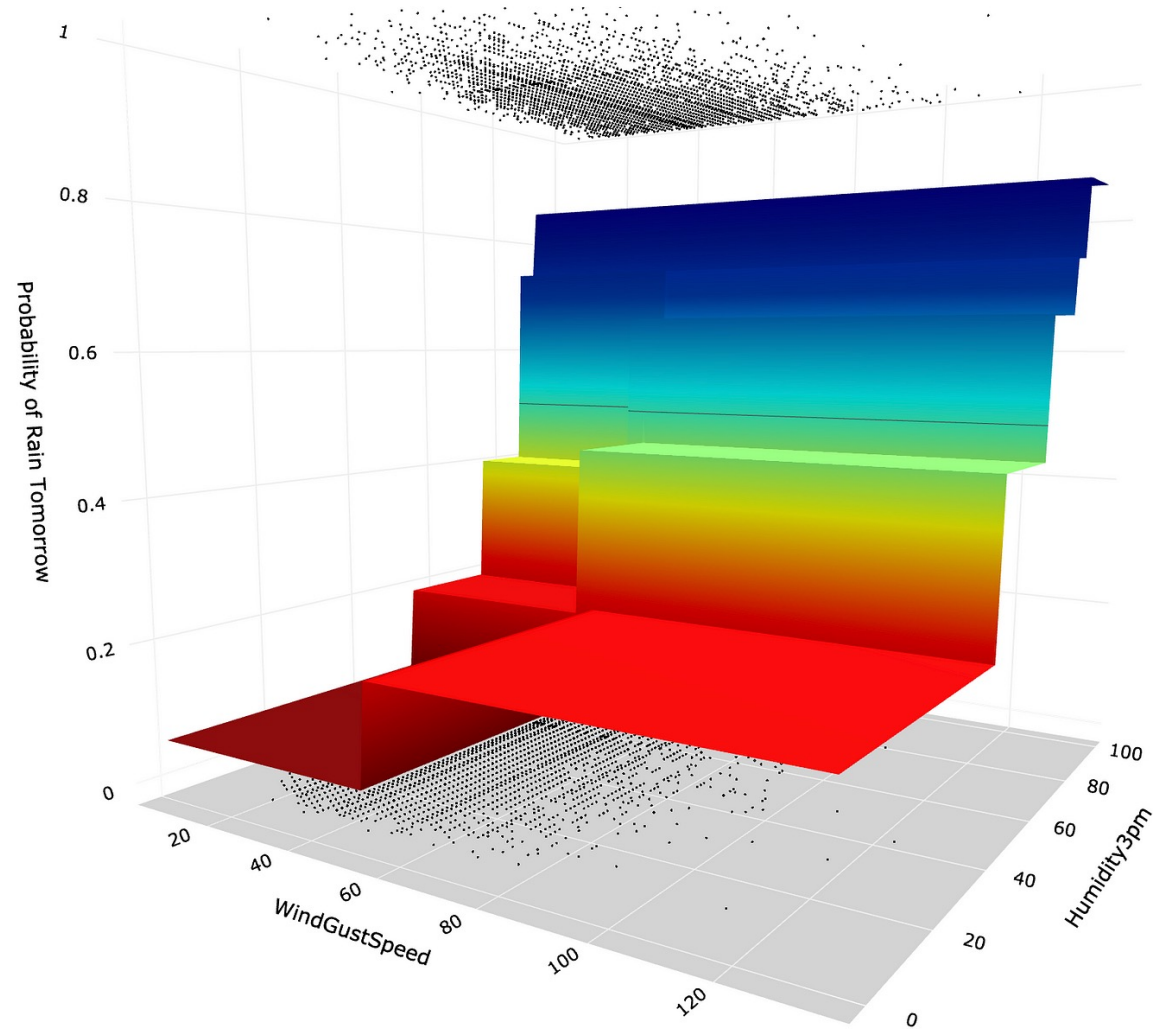
- To identify the best split, you would have to follow all the same steps outlined above. The split with the lowest entropy is the best one. Similarly, if the entropy of the two child nodes is not lower than that of a parent node, you should not split any further.



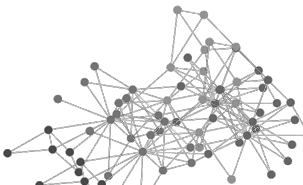
Another example: Rain prediction



Prediction plane using decision tree



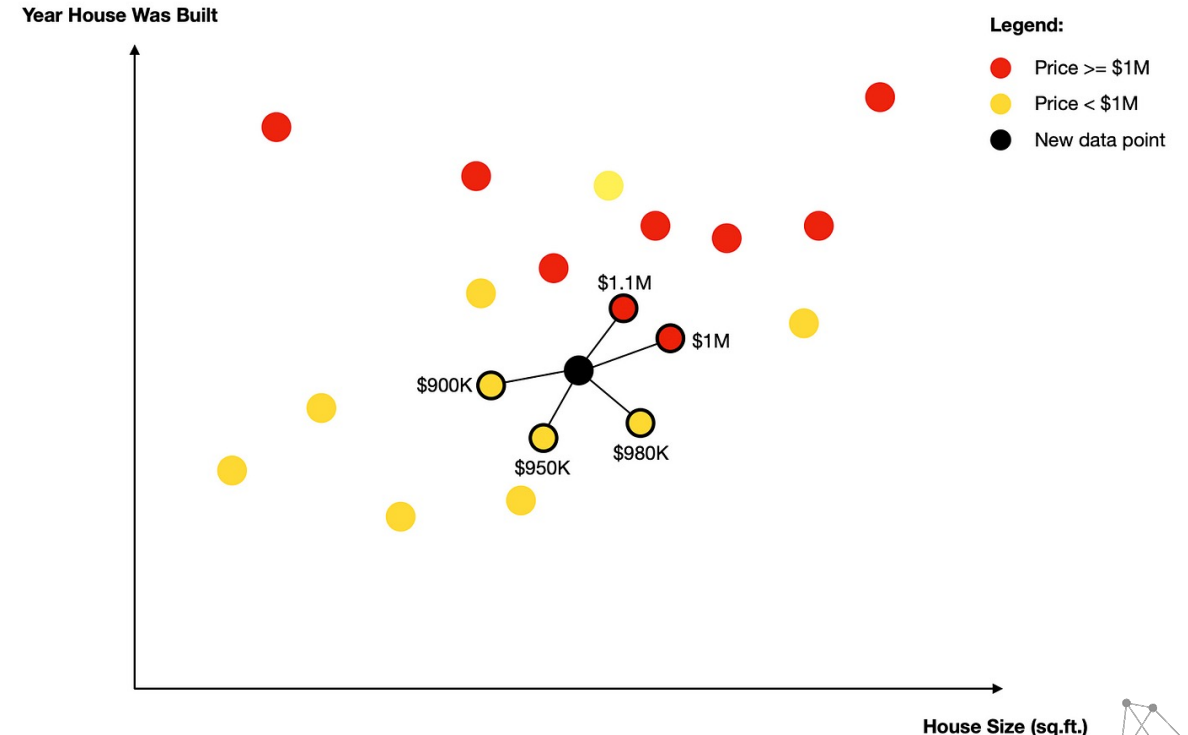
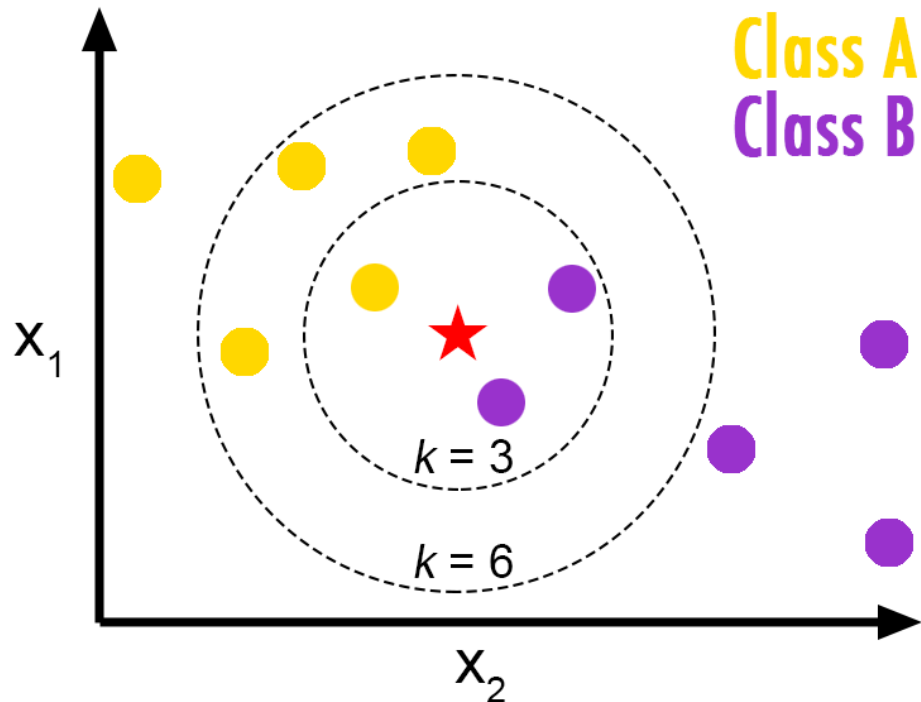
Jackson, L. (2018, November 29). *CART classification and regression trees for clean but powerful models*. Towards Data Science. Retrieved from <https://medium.com/towards-data-science/cart-classification-and-regression-trees-for-clean-but-powerful-models-cc89e60b7a85>



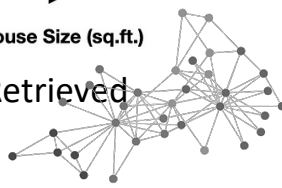
Section two



K-Nearest Neighbors (KNN)



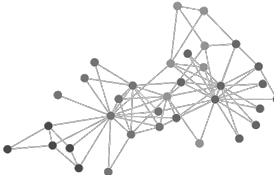
Ramírez, D. (2018, October 23). *K-nearest neighbors (KNN): How to make quality predictions with supervised learning*. Towards Data Science. Retrieved from [<https://towardsdatascience.com/k-nearest-neighbors-knn-how-to-make-quality-predictions-with-supervised-learning-d5d2f326c3c2>]



How does kNN work?

- The algorithm makes its predictions based on the nearest neighbours, we need to tell the algorithm the exact number of neighbors we want to consider. Hence, “k” represents the number of neighbors and is simply a hyperparameter that we can tune.

$$d(x,y) = \sqrt{\sum_{i=1}^n (y_i - x_i)^2}$$



Example: Data

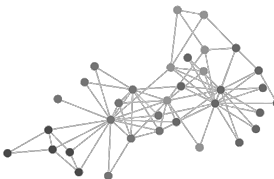
No		X1 transaction date	X2 house age	X3 distance to the nearest MRT station	X4 number of convenience stores	X5 latitude	X6 longitude	Y house price of unit area
0	1	2012.917	32.0	84.87882	10	24.98298	121.54024	37.9
1	2	2012.917	19.5	306.59470	9	24.98034	121.53951	42.2
2	3	2013.583	13.3	561.98450	5	24.98746	121.54391	47.3
3	4	2013.500	13.3	561.98450	5	24.98746	121.54391	54.8
4	5	2012.833	5.0	390.56840	5	24.97937	121.54245	43.1
...	...	...	...	...	...	...	...	...
409	410	2013.000	13.7	4082.01500	0	24.94155	121.50381	15.4
410	411	2012.667	5.6	90.45606	9	24.97433	121.54310	50.0
411	412	2013.250	18.8	390.96960	7	24.97923	121.53986	40.6
412	413	2013.000	8.1	104.81010	5	24.96674	121.54067	52.5
413	414	2013.500	6.5	90.45606	9	24.97433	121.54310	63.9

414 rows × 8 columns



KNN can be used as:

- **Classification:** the algorithm uses simple majority voting to assign the label to the new data point. In our example, the majority consists of 3 neighbors with a price < \$1M. Hence, the predicted label for the new data point is < \$1M.
- **Regression:** the algorithm calculates the average value of all the neighbors. In our example this would be: $(\$900K + \$950K + \$980K + \$1M + \$1.1M) / 5 = \$986K$. So, the predicted price of a house (new data point) is \$986K.



Majority Voting in KNN

- The KNN algorithm classifies the new data point based on the labels of its **K nearest neighbors**. The label is assigned by **majority vote**.
- For example, if $K=5$, the algorithm will look at the 5 nearest data points in the training set. It will then count how many of these neighbors belong to each class (class 0 or class 1). The new data point will be assigned the label of the class that appears the most frequently among the K neighbors.

Majority Voting Formula

Let's say N_0 is the number of neighbours that belong to class 0, and N_1 is the number of neighbours that belong to class 1. So, we have:

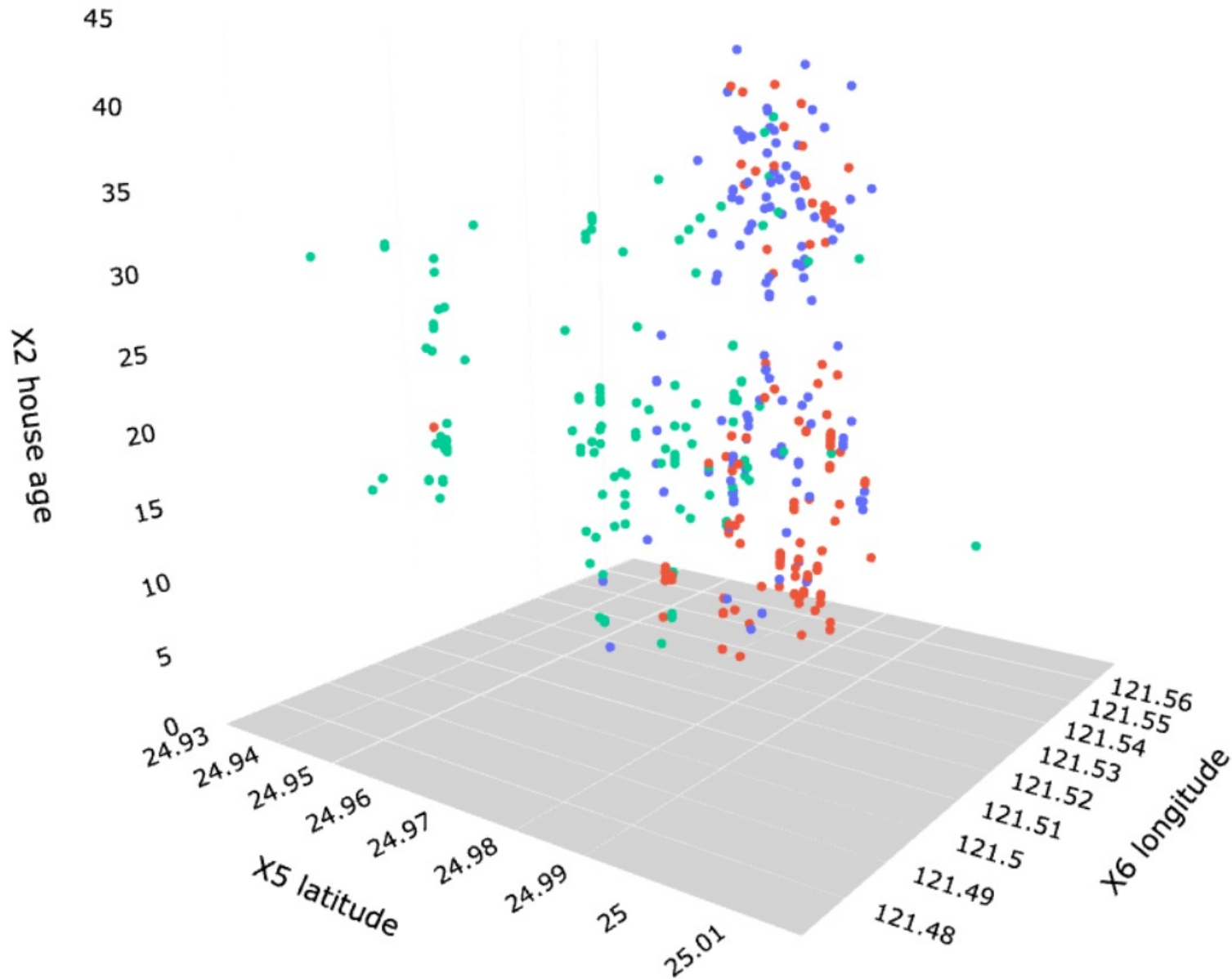
$$K = N_0 + N_1$$

The predicted class $\hat{y}$ for the new data point x_{new} is given by:

$$\hat{y} = \begin{cases} 0, & \text{if } N_0 > N_1 \\ 1, & \text{if } N_1 > N_0 \end{cases}$$

If $N_0 > N_1$, the label assigned will be class `0`.

If $N_1 > N_0$, the label assigned will be class `1`.



KNN regression

- Result of regression



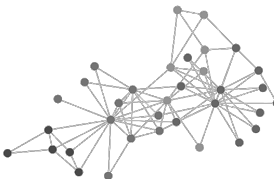
Section three





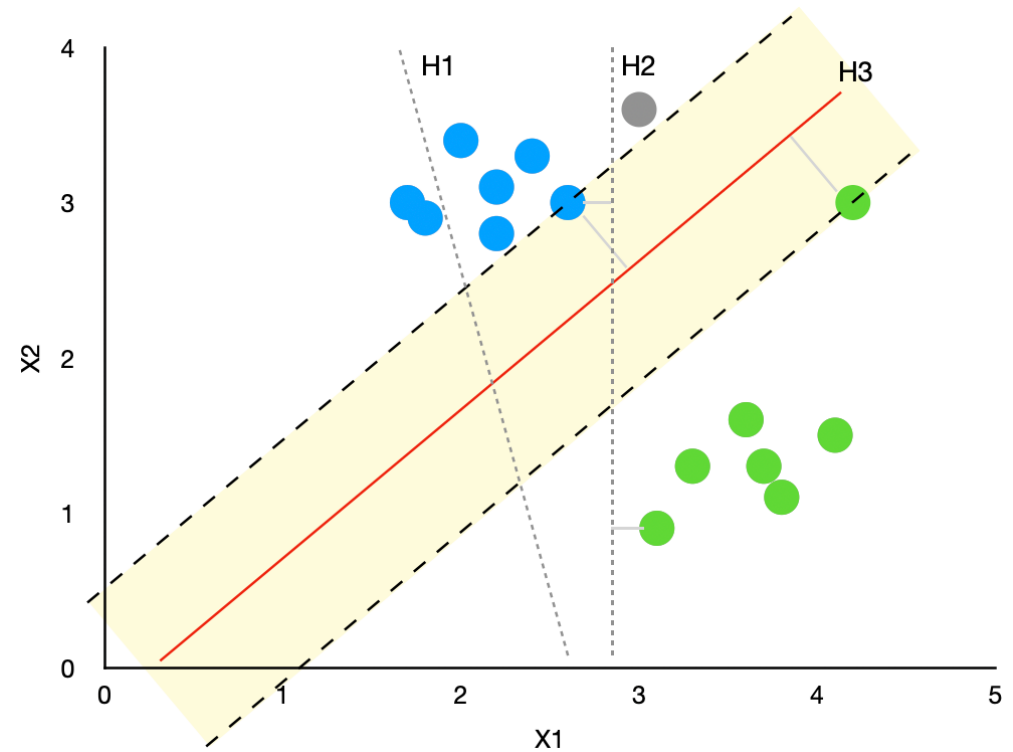
SVM classification algorithm — a brief explanation

- Let's assume we have a set of points that belong to two separate classes. We want to separate those two classes in a way that allows us to correctly assign any future new points to one class or the other.
- SVM algorithm attempts to find a hyperplane that separates these two classes with the highest possible margin. If classes are fully linearly separable, a **hard-margin** can be used. Otherwise, it requires a **soft-margin**.



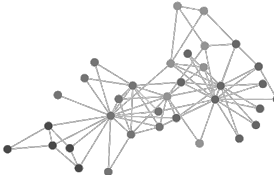
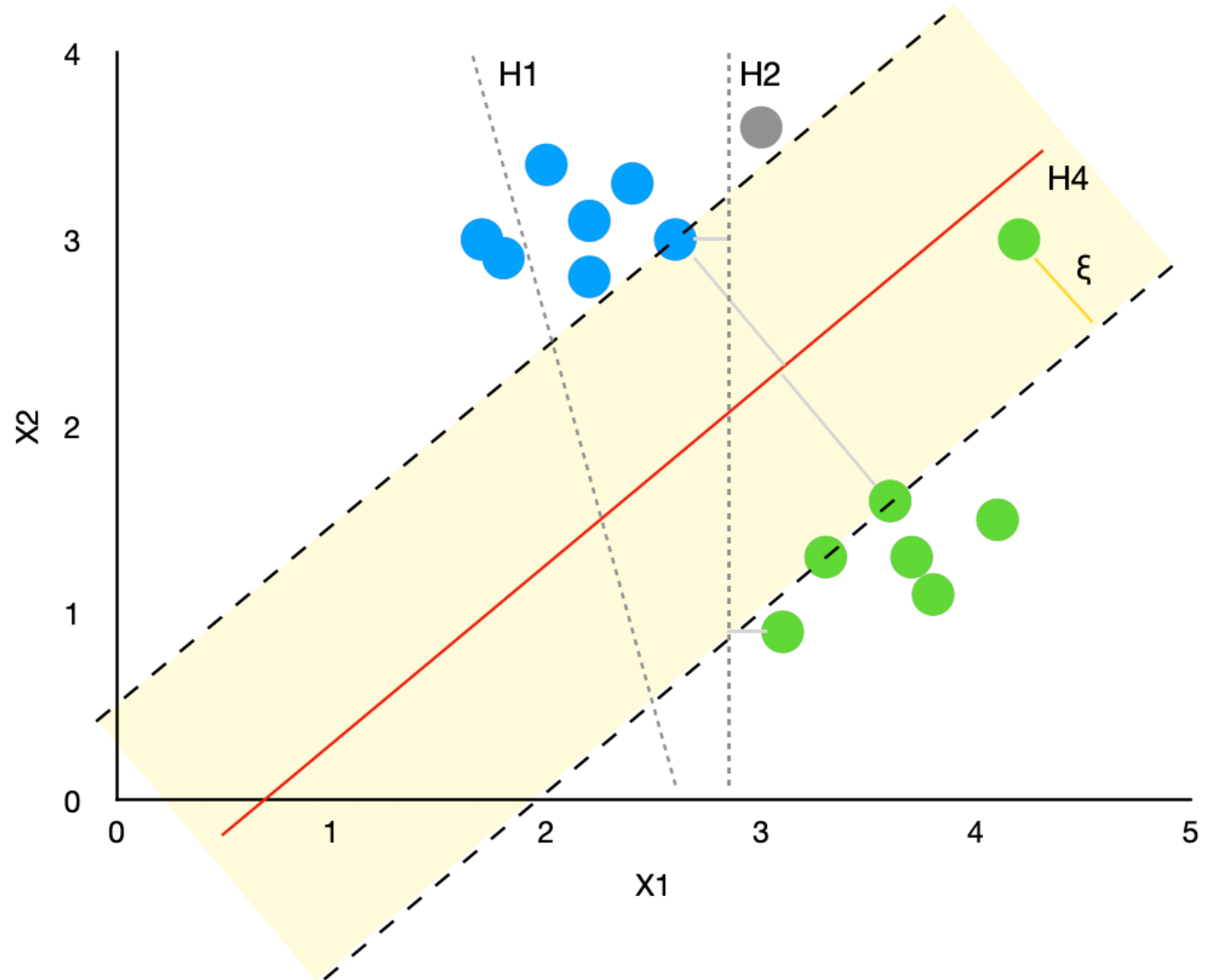
Hard margin

- Hyperplane called “**H1**” cannot accurately separate the two classes; hence, it is not a viable solution to our problem.
- The “**H2**” hyperplane separates classes correctly. However, the margin between the hyperplane and the nearest blue and green points is tiny. Hence, there is a high chance of incorrectly classifying any future new points. E.g., the new grey point ($x_1=3$, $x_2=3.6$) would be assigned to the green class by the algorithm when it is obvious that it should belong to the blue class instead.
- Finally, the “**H3**” hyperplane separates the two classes correctly and with the highest possible margin (yellow shaded area). Solution found!



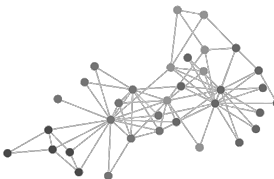
Soft margin

- Sometimes, it may not be possible to separate the two classes perfectly. In such scenarios, a **soft-margin** is used where some points are allowed to be misclassified or to fall inside the margin (yellow shaded area).



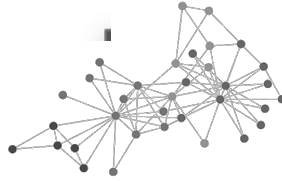
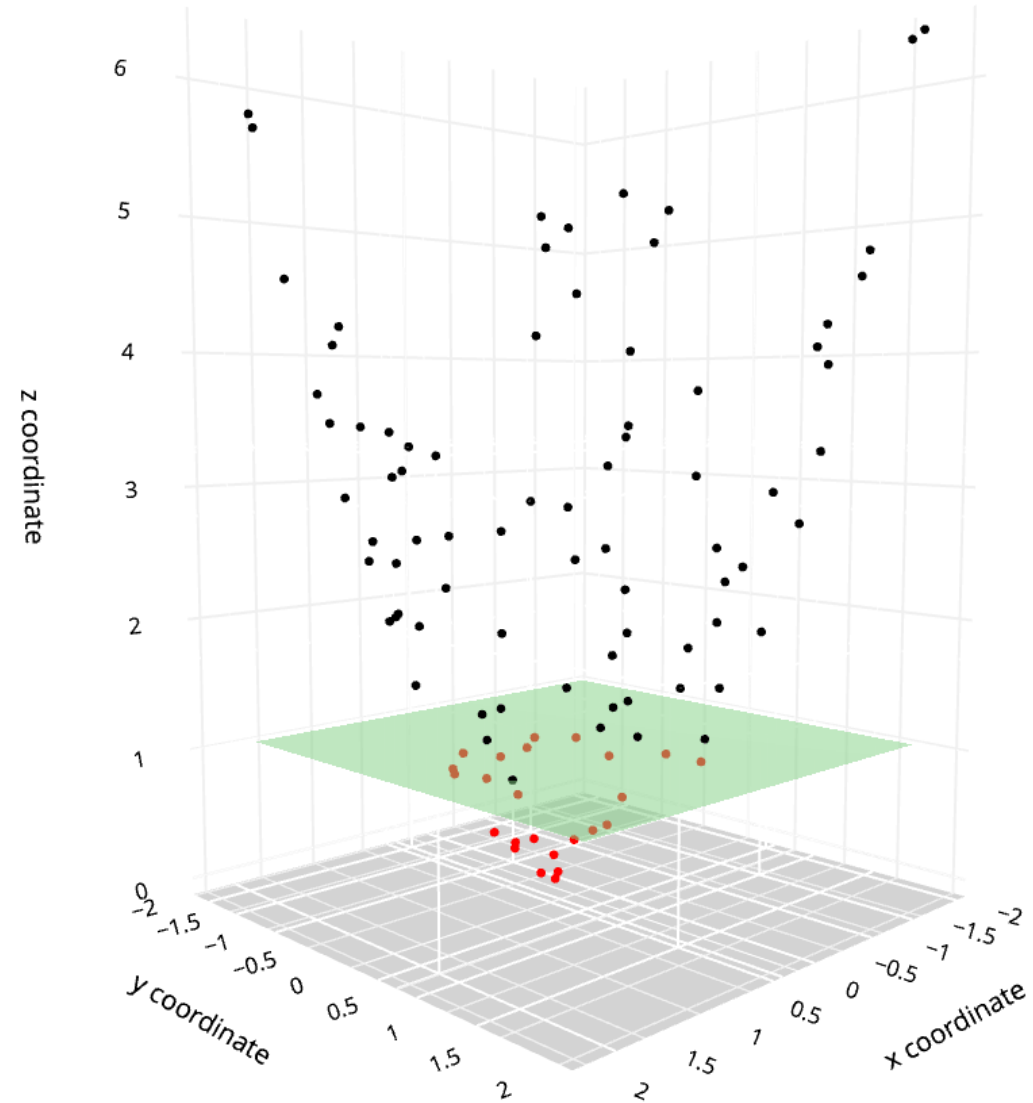
Kernel trick

- The above explanation of SVM covered examples where blue and green classes are linearly separable. However, what if we wanted to apply SVMs to non-linear problems? How would we do that?
- This is where the kernel trick comes in. A **kernel is a function** that takes the original non-linear problem and transforms it into a linear one within the higher-dimensional space. To explain this trick, let's study the below example.



How to transform this problem into a linear one?

$$z = x^2 + y^2$$

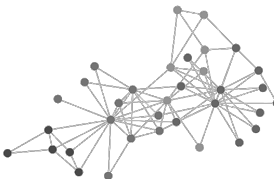


Radial Basis Function (RBF) kernel

$$K(x, x') = e^{-\gamma ||x - x'||^2}$$

$$\gamma = \frac{1}{n \text{ features} * \sigma^2}$$

- $||x - x'||^2$ is the squared Euclidean distance between two feature vectors (2 points). **Gamma** is a scalar that defines how much influence a single training example (point) has.

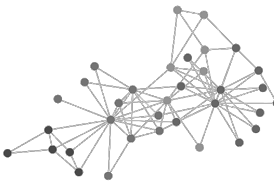


Example: Data

	id	rated	victory_status	winner	white_id	white_rating	black_id	black_rating	opening_eco	rating_difference	white_win
0	TZJHLlJE	False	outoftime	white	bourgris	1500	a-00	1191	D10	309	1
1	l1NXvwaE	True	resign	black	a-00	1322	skinnerua	1261	B00	61	0
2	mIICvQHh	True	mate	white	ischia	1496	a-00	1500	C20	-4	1
3	kWKvrqYL	True	mate	white	daniamurashov	1439	adivanov2009	1454	D02	-15	1
4	9tXo1AUZ	True	mate	white	nik221107	1523	adivanov2009	1469	C41	54	1
...	...	...	...	...	...	...	...	...	...	...	...
20053	EfqH7VVH	True	resign	white	belcolt	1691	jamboger	1220	A80	471	1
20054	WSJDhbPI	True	mate	black	jamboger	1233	farrukhasomiddinov	1196	A41	37	0
20055	yrAas0Kj	True	mate	white	jamboger	1219	schaaksmurf3	1286	D00	-67	1
20056	b0v4tRyF	True	resign	white	marcodisogno	1360	jamboger	1227	B07	133	1
20057	N8G2JHGG	True	mate	black	jamboger	1235	ffbob	1339	D00	-104	0

20058 rows x 11 columns

chess games data from Kaggle



Code

```
import pandas as pd # for data manipulation
import numpy as np # for data manipulation

from sklearn.model_selection import train_test_split # for splitting the data i
from sklearn.metrics import classification_report # for model evaluation metric
from sklearn.svm import SVC # for Support Vector Classification model

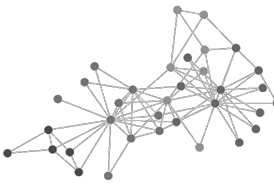
import plotly.express as px # for data visualization
import plotly.graph_objects as go # for data visualization
```

```
# Read in the csv
df=pd.read_csv('games.csv', encoding='utf-8')

# Difference between white rating and black rating - independent variable
df['rating_difference']=df['white_rating']-df['black_rating']

# White wins flag (1=win vs. 0=not-win) - dependent (target) variable
df['white_win']=df['winner'].apply(lambda x: 1 if x=='white' else 0)

# Print a snapshot of a few columns
df.iloc[:,[0,1,5,6,8,9,10,11,13,16,17]]
```



Training

```
def fitting(X, y, C, gamma):
    # Create training and testing samples
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,

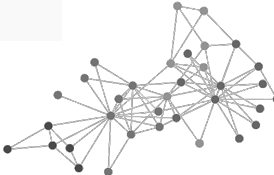
    # Fit the model
    # Note, available kernels: {'linear', 'poly', 'rbf', 'sigmoid', 'precompu
    model = SVC(kernel='rbf', probability=True, C=C, gamma=gamma)
    clf = model.fit(X_train, y_train)

    # Predict class labels on training data
    pred_labels_tr = model.predict(X_train)
    # Predict class labels on a test data
    pred_labels_te = model.predict(X_test)

    # Use score method to get accuracy of the model
    print('----- Evaluation on Test Data -----')
    score_te = model.score(X_test, y_test)
    print('Accuracy Score: ', score_te)
    # Look at classification report to evaluate the model
    print(classification_report(y_test, pred_labels_te))
    print('-----')

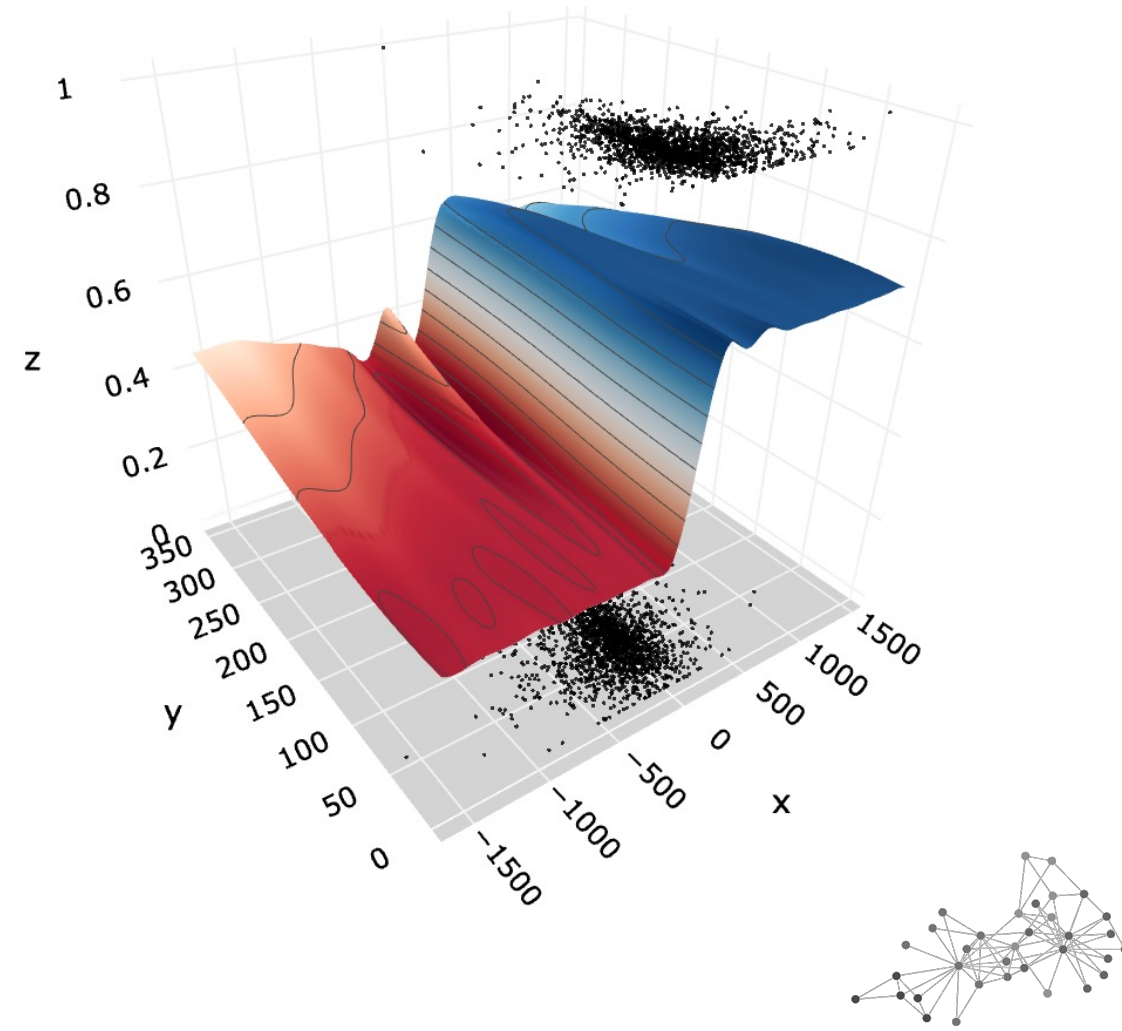
    print('----- Evaluation on Training Data -----')
    score_tr = model.score(X_train, y_train)
    print('Accuracy Score: ', score_tr)
    # Look at classification report to evaluate the model
    print(classification_report(y_train, pred_labels_tr))
    print('-----')

    # Return relevant data for chart plotting
    return X_train, X_test, y_train, y_test, clf
```



Result

Note, black points at the top are actual class=1 (white won), and the ones at the bottom are actual class=0 (white did not win). Meanwhile, the surface is the probability of a white win produced by the model.



Conclusion

- **Decision Trees** offer a clear and interpretable model structure that is easy to understand and visualize. They are effective for both classification and regression tasks but can be prone to overfitting, especially with complex datasets. Pruning techniques and ensemble methods like Random Forests help mitigate this issue and enhance their predictive power.
- **K-Nearest Neighbors (KNN)** is a simple and intuitive instance-based learning algorithm that excels in tasks where the decision boundary is irregular. While KNN is easy to implement and understand, it can be computationally expensive with large datasets and sensitive to the choice of K and the distance metric. Dimensionality reduction techniques can be applied to improve its efficiency.
- **Support Vector Machines (SVM)** are robust and effective for classification tasks, particularly when dealing with high-dimensional spaces and complex decision boundaries. SVMs are capable of handling both linear and non-linear data by using kernel functions. However, SVMs can be sensitive to the choice of parameters and may require careful tuning.



Thanks
for
attention

